Linked list

```
Class Node {
    Node next;
    int data;
}

Class Linked {
    Node root;

    Public Linked() {
        root = null;
    }
    Public Node getNewNode(int key) {
        Node a = new Node();
        a.next = null;
        a.data = key;
        return a;
    }
    Public Node insert(Node node, int key) {
        if(node == null)
            return getNewNode(key);
        else
            node.next = insert(node.next, key);
        return node;
    }
}
Public Class LinkedListAPP {
    P S V M(String[] args) {
        Node root = null;
        Linked a = new Linked();
        root = a.insert(root, 1);
        root = a.insert(root, 2); root = a.insert(root, 3);
    }
}
```

// Printing Linked List:

pass here root node.

```
Public void printList(Node node) {
    if(node == null)
        return;
    S.o.p(node.i + " ");
    printList(node.next);
}
```

// Insert an element at begining.

```
public Node insertFront(int i, Node node) {
    Node a = getNewNode(i);
    a.next = node;
    return a;
}
```

// Insert element at given position

```
public Node insertAtPosition(int i, int position, Node node)
{
    if(position < 0) {
        S.o.p("pos. can't be less than 1");
        return node;
    }
    if(node == null && position > 1) {
        S.o.p("pos is greater than element exists");
        return node;
    }
```

```
if (node == null && position == 1) {{
    return getNewNode(i);
}

if (position == 1) {
    Node newNode = getNewNode(i);
    newNode.next = node;
    return newNode;
}

node.next = insertAtPosition(i, position-1, node.next);
return node;

}
```

// delete.

```
public Node delete (Node node) {
    if (node == null || node.next == null) {
        return null;
    }
    Node tmp = node;
    while (tmp.next.next != null)
        tmp = tmp.next;

    tmp.next = null;
    return node;
}
```

// delete front.

```
public Node deletefront (Node node) {

    if (node == null)
        return node;

    return node.next;
}
```

// Delete element at given position

```
public Node deleteAtPosition (int position, Node node) {
    if (position < 0) {
        S.o.p ("not a valid position");
        return node;
    }
```

```
if (node == null && position > 1) {
        S.O.P("not a valid position");
        return node;
}

if (position == 1)
        return node.next;

node.next = deleteAtPosition(position-1, node.next);
return node;

}
```

// Size.

```
Public int getSizeOfList(Node node) {

        if (node == null)
                return 0;

        return getSizeOfList(node.next) + 1;

}
```

// Search node in Linked List.

```
public boolean isNodeExists(int val, Node node) {

    if (node == null)
        return false;

    while (node != null) {
        if (node.data == val)
            return true;
        else
        node = node.next;
    }
    return false;

}
```

---

* Return node where cycle begins.

```
while( fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast)
        break;
}
if( slow != fast)
    return null;
slow = head;
while( slow != fast) {
    slow = slow.next;
    fast = fast.next;
}
return slow;
```

// Rotate the Linked List in clock-wise by K nodes.

12 → 99 → 37 → 8 → 18 → null

K=1 : 18 → 12 → 99 → 37 → 8 → null.

(K=2) → 8 → 18 → 12 → 99 → 37 → null.

```
Public Node rotateClockWise(int K, Node node) {

    if(node == null || K < 0)
        return node;

    int sizeOfLinkedList = getSizeOfList(node);
    K = K % sizeOfLinkedList;

    if(K == 0)
        return node;
    Node tmp = node;
    int i = 1;
    while(i < sizeOfLinkedList - K) {
        tmp = tmp.next;
        i++;
    }
    Node temp = tmp.next;
    Node head = temp;
    tmp.next = null;

    while(temp.next != null) {
        temp = temp.next;
    }
    temp.next = node;
    return head;

}
```

// Rotate the Linked List in Anti-Clockwise by K-nodes.

```
Public Node rotateAntiClockwise(int K, Node node) {
    if (node == null || K < 0) {
        return node;
    }
    int sizeOfLinkedList = getSizeOfList(node);
    K = K % sizeOfLinkedList;

    if (K == 0)
        return node;

    Node tmp = node;
    int i = 1;

    while (i < K) {
        tmp = tmp.next;
        i++;
    }
    Node temp = tmp.next;
    Node head = temp;
    tmp.next = null;
    i = 1;

    while (temp.next != null) {
        temp = temp.next;
    }
    temp.next = node;
    return head;

}
```

// Reverse the LinkedList — pass here head

```
Public Node reverse(Node node) {
    if (node == null || node.next == null)
        return node;

    Node temp = reverse(node.next);
    node.next.next = node;
    node.next = null;

    return temp;
}
```

// Get middle node of linked list

```
Public Node middleNode(Node node) {
    if (node == null)
        return null;

    Node a = node;
    Node b = node;

    while(b != null && b.next != null) {
        a = a.next;
        b = b.next.next;
    }
    return a;
}
```

b != null && b.next != null नहीं काम कर रहा है error दे रहा है Null Pointer exception (बाद में चेक करो).

// merge two linked list in such a way that the element is in increasing order.

// first sort them here using mergesort.

```
public Node mergeSort(Node node) {
    if (node == null || node.next == null)
        return node;

    Node middle = middleNode(node);
    Node secondHalf = middle.next;
    middle.next = null;

    return merge(mergeSort(node), mergeSort(secondHa
}

public Node merge(Node a, Node b) {
    Node temp = new Node();
    Node finalList = temp;

    while (a != null && b != null) {
        if (a.i < b.i) {
            temp.next = a;
            a = a.next;
        }
        else {
            temp.next = b;
            b = b.next;
        }
        temp = temp.next;
    }
    temp.next = (a == null) ? b : a;
    return finalList.next;
}
```

```
public Node mergeList(Node a, Node b) {
    (a = b = node)
    while (b.next != null && b.next.next != null) {
        a = a.next;
        b = b.next.next;
    }
    return merge(mergeSort(a), mergeSort(b))
}
```

To delete linked list simply put head = null in Java.

// Remove Duplicate elements from a sorted list.

```
Public Node removeDuplicateSortedList(Node node) {

    if(node == null || node.next == null)
        return node;

    if(node.i == node.next.i) {
        node.next = node.next.next;
        removeDuplicateSortedList(node);
    }
    else
        removeDuplicateSortedList(node.next);
    return node;

}
```

// delete N nodes after M Nodes

```
Public Node deleteNNodesPostMNodes(int n, int m, Node node)
{
    if(node == null)
        return node;
    Node temp1 = node;  Node temp2 = node;
    int i=0, j=0;
    while(i++ < m-1)
        temp1 = temp1.next;
    temp2 = (m==0) ? temp1 : temp1.next;

    while(j++ < n) {
        if(temp2 == null){S.o.p("not enough element to delete");
```

```
            return node;
        }
        temp2 = temp2.next;
    }

    if (m == 0)
        return temp2;
    else
        temp1.next = temp2;

    return node;

}
```

// Remove Duplicate items from unsorted linked list using HashMap.

```
public Node removeDuplicateUsingHashMap(Node node) {
    if (node == null)
        return null;
    HashMap<Integer, Integer> s = new HashMap<>();
    Node head = node;
    Node prev = null;
    while(node != null) {
        if(!s.containsKey(node.i)) {
            s.put(node.i, 1);
            prev = node; node = node.next;
        } else {
            prev.next = node.next; node = node.next;
        }
    }
    return head;
}
```

// Check Loop is present in LinkedList

head

```
Public boolean ifLoopPresent(Node node) {
    if(node == null)
        return false;
    Node slow, fast;
    slow = fast = node;
    while(fast.next != null && fast.next.next != null){
        slow = slow.next;
        fast = fast.next.next;
        if(slow == fast)
            return true;
    }
    return false;
}
```

// Length of the loop

head

slow == fast

```
Public int lengthOfLoop(Node node) {
    boolean loop = ifLoopPresent(node);
    if(loop) {
        while(slow.next != fast) {
            slow = slow.next;
            length++;
        }
    }
    return length;
}
```

यहाँ पर हम गलत कर दिये पहले हमे loop निकालना होगा उसके बाद फिर हम length निकालेंगे.

// Detect and Remove Loop

if Last node point to head

```
if (slow == fast) {
    if (fast == head) {
        while (slow.next != head)
            slow = slow.next;
        slow.next = null;
    } else {
        slow = head;
        while (slow.next != fast.next) {
            slow = slow.next; fast = fast.next; }
        fast.next = null;
    }
}
```

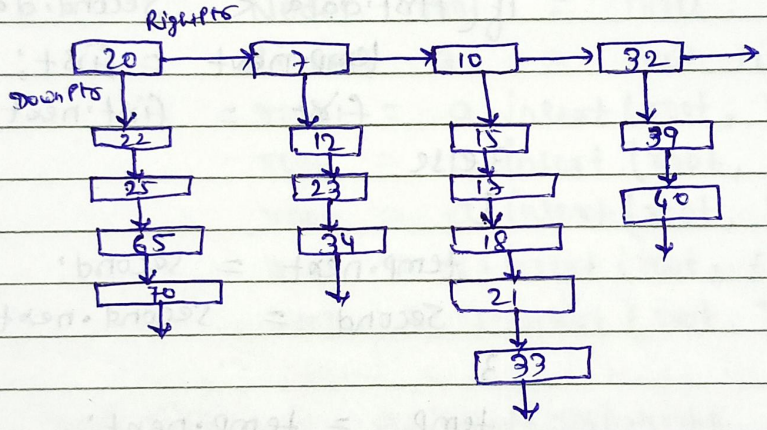
// Get Kth Last Node of Linked List.

```
Public int returnKthLastNode(Node node, int K) {
    if (node == null)
        return -1;
    Node Ptr1, Ptr2;
    Ptr1 = Ptr2 = node;
    for (int i = 1; i < K; i++) {
        Ptr2 = Ptr2.next;
        if (Ptr2 == null) {
            S.O.P("K is greater than total node present in list");
            return -1;
        }
    }
    while (Ptr2.next != null) {
```

```
        Ptr1 = Ptr1.next;
        Ptr2 = Ptr2.next;
    }
    return Ptr1.data;
}
```

3.

// Flatten a Sorted multilevel linked list




```
class Node {
    Node right;
    Node next;
    int data;
}

class LinkedList {

    Node root;
    Public LinkedList() {
        root = null;
    }
```

```
Public Node flatten(Node node){
    if(node == null || node.right == null)
        return node;
    return merge(node, flatten(node.right));
}

Public Node merge(Node first, Node second){
    Node temp = new Node();
    Node finalList = temp;

    while(first != null && second != null){
        if(first.data < second.data){
            temp.next = first;
            first = first.next;
        }else
        {
            temp.next = second;
            second = second.next;
        }
        temp = temp.next;
    }
    temp.next = (first != null)? first : second;
    finalList.next.right = null;
    return finalList.next;
}
Public Node getNewNode(int key){
    Node a = new Node();
    a.next = null;
    a.right = null;
    a.data = key;
    return a;
}
```

```
    Public Node insert(Node node, int key) {
        if ( node == null)
            return getNewNode(key);

        node.next = insert(node.next, key);
        return node;

    }

}
Public class LinkedListFlatten {
    p s v m (String[] args) {

        Node root = null;
        Linked a = new Linked();
        root = a.insert(root, 20);
        root = a.insert(root, 22);
        root = a.insert(root, 25);
        root = a.insert(root, 65);
        root = a.insert(root, 70);

    root.right = a.insert(root.right, 7);
        "      =                    , 12
        "      =                      23
        "      =                      34
                                        , 15, 17, 18, 21, 33
     root.right.right = a.insert(root.right.right, 10)
                                        , 39, 40
     root.right.right.right = a.insert(root.r.r.r, 32);

     Node flattenList = a.flatten(root);
     a.printLinkedList(flattenList);

    }
}
```

~~Stack~~

// move all occurrences of an element to the end of list.

// Rearrange list in zig-zag manner

A<B>C<D>E<F>G

head

2 → 3 → 5 → 9 → 11 → 12

2 → 5 → 3 → 11 → 9 → 12

```
public Node rearrange (Node head) {

    if(head == null || head.next == null)
        return head;

    Node node = head;
    boolean flag = true;

    while (node.next != null) {
        if(flag) {
            if( node.data > node.next.data) {
                int t = node.data;
                node.data = node.next.data;
                node.next.data = t;
            } flag = !flag; }
        else {
            if(node.data < node.next.data)
                swap
            flag = !flag;
        }
        node = node.next;
    }
    return head;
}
```

// Check if a linked list of String forms a palindrome.
// Delete last occurrence of an element.
// Length of longest palindrome list in a linked list.
// Compare two strings represented as linked list.

// Add 1 to the linked list.

1 → 9 → 9 → 9 → 9 ⇒ 2 → 0 → 0 → 0 → 0

```
Public int addOneToList(Node node) {

    if(node == null)
        return 1;

    int res = node.data + addOneToList(node.next);
    node.data = res % 10;
    return res/10;
}

Public Node addOne(Node node) {
    int c = addOneToList(node);
    if(c == 1) {
        Node head = createNewNode(1);
        head.next = node;
        return head;
    }
    return node;
}
```

// Add two numbers represented as linked list through recursion.

```
int carry;
Node result, tempNode;
public void addSumOfTwoLists(Node node1, Node node2)

    int count1 = 0, count2 = 0;
    Node head1 = node1, head2 = node2;

    while(node1 != null) {
        count1++;
        node1 = node1.next;
    }

    while(node2 != null) {
        count2++;
        node2 = node2.next;
    }
    if(count1 == count2) {
        calSum(head1, head2);
    }
    else {
        if(count1 < count2) {
            Node tmp = head1;
            head1 = head2;
            head2 = tmp;
        }

        int d = Math.abs(count2 - count1);
        node1 = head1;
        node2 = head2;
```

```
        while(d > 0) {
            node1 = node1.next;
            tempNode = node1;
            d--;
        }

        calSum(node1, node2);
        addRestSum(head1);
    }
    if(carry > 0) {

        Node a = getNewNode(carry);
        a.next = result;
        result = a;
    }
}

public void calSum(Node node1, Node node2) {

    if(node1 == null)
        return;
    calSum(node1.next, node2.next);
    int s = node1.data + node2.data + carry;

    carry = s/10;
    if(result == null)
        result = getNewNode(s % 10);
    else {
        Node a = getNewNode(s % 10);
        a.next = result;
        result = a;
    }
}
```

```
Public void addRestSum(Node node) {

    if(node == null)
        return;

    if( node != tempNode) {
        addRestSum(node.next);
        int s = node.data + carry;
        carry = s/10;
        Node a = getNewNode(s %. 10);
        a.next = result;
        result = a;
    }
}
```

// Add two number represented as linked list using reverse.

```
Public Node addTwoNumberss(Node node1, Node node2){

    node1 = reverse(node1);
    node2 = reverse(node2);

    Node newListHead = null;
    Node Prev = null;
    int Sum, c = 0;

    while(node1 != null || node2 != null) {
        Sum = c + (node1 != null ? node1.data:0)
              + (node2 != null ? node2.data :0);
        c = Sum / 10;
        Node node = getNewNode(Sum %. 10);
```

```
        if (newListHead == null)
            newListHead = node;
        else
            prev.next = node;

        prev = node;

        if (node1 != null)
            node1 = node1.next;

        if (node2 != null)
            node2 = node2.next;
    }
    if (c != 0)
        prev.next = getNewNode(c);

    newListHead = reverse(newListHead);

    return newListHead;
}

// Substract two numbers represented as linked list

Public Node SubstractTwoNumbers(Node node1, Node node2)
    Node head1 = node1;
    Node head2 = node2;

    int count1 = 0, count2 = 0;

    while(node1 != null) {
        node1 = node1.next;
        count1++;
    }
```

```
while(node2 != null) {
    node2 = node2.next;
    count2++;
}

node1 = head1;
node2 = head2;

if((count1 < count2) || (count1 == count2 &&
    node2 == getBiggestList(node1, node2))) {
        Node t = node1;
        node1 = node2;
        node2 = t;
}
node1 = reverse(node1);
node2 = reverse(node2);

Node newListHead = null;
Node prev = null;
int diff;
boolean borrow = false;

while(node1 != null || node2 != null) {
    if(borrow) {
        node1.data = node1.data - 1;
        borrow = false;
    }

    if(node1 != null && node2 != null &&
        node1.data < node2.data) {
        node1.data = node1.data + 10;
        borrow = true;
    }
```

```
diff = (node1 != null ? node1.data : 0) - (node2 != null ? node2.data : 0);

Node node = getNewNode(diff);
if(newListHead == null)
    newListHead = node;
else
    Prev.next = node;

Prev = node;

if(node1 != null)
    node1 = node1.next;

if(node2 != null)
    node2 = node2.next;

}
newListHead = reverse(newListHead);
return newListHead;
}
Public Node getBiggerList(Node node1, Node node2) {
    Node head1 = node1, head2 = node2;
    while(node1 != null) {
        if(node1.data > node2.data)
            return head1;
        else if(node1.data < node2.data)
            return head2;

        node1 = node1.next; node2 = node2.next;
    }
    return head1;
}
```

// Delete nodes which have greater values on the right

```
public Node deleteGreaterValuesOnRight(Node node){

    if(node == null || node.next == null)
        return node;

    Node reverse = reverse(node);
    Node tmp = reverse;
    int max = tmp.data;

    while(tmp.next != null) {
        if(tmp.next.data > max) {
            max = tmp.next.data;
            tmp = tmp.next;
        }
        else
            tmp.next = tmp.next.next;
    }
    node = reverse(reverse);

    return node;
}
```

// Check Linked list are identical or not.

```
while(node1 != null && node2 != null) {
    if(node1.data != node2.data)
        return false;
    node1 = node1.next; node2 = node2.next;
}
return (node1 == null) && node2 == null;
```

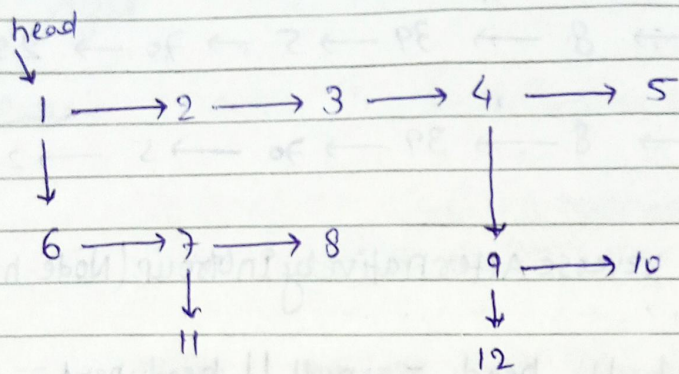
// Reverse List alternatively in Group of K elements.

12 → 99 → 8 → 39 → 5 → 70 → 25

99 → 12 → 8 → 39 → 70 → 5 → 25

```
Public Node reverseAlternativelyInGroup(Node head, int K){

    if( K <= 1 || head == null || head.next == null)
        return head;

    Node prev, next;
    Prev = next = null;
    Node node = head;
    int i = 0;

    While( node != null && i< K) {
        next = node.next;
        node.next = prev;
        prev = node;
        node = next;
        i++;
    } i = 0;
    if( next != null) {
        head.next = next; node = next;
    while(node != null && i< K-1) {
            node = node.next;
            i++;
    }
    if( node != null)
        node.next = reverseAlternativelyInGroup(node.next, K);
    return prev;
}
```

// Flaten a multi level linked list level wise



1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12

```
Class Node {
    Node next;
    Node child;
    int data;
}

Class LinkedListApp {
    public Node createNewNode(int i) {
        Node a = new Node();
        a.data = i;
        a.next = null;
        a.child = null;
        return a;
    }

    Public Node insert(Node node, int i) {
        if(node == null)
            return createNewNode(i);
        node.next = insert(node.next, i);
        return node;
```

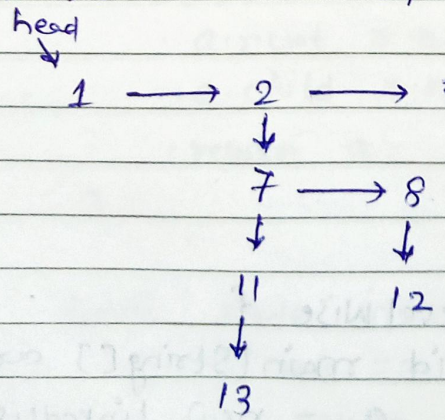
```
Public Node createList(int arr[]) {
    Node node = null;
    for(int i = 0; i < arr.length; i++)
        node = insert(node, arr[i]);
    return node;
}
Public Node flatenEasy(Node node) {
    if(node == null)
        return node;
    Node start, end;
    start = end = node;

    while(end.next != null)
        end = end.next;

    while(start != null) {
        if(start.child != null) {
            end.next = start.child;
            end = end.next;
            while(end.next != null)
                end = end.next;
        }
        start = start.next;
    }
    return node;
}
}

Public class FlattenAPPLevelWise {
    Public static void main(String[] args) {
        LinkedListApp a = new LinkedListApp();
        Node root1, root2, root3, root4, root5;
```

```
int arr1[] = new int[] {1, 2, 3, 4, 5};
int arr2[] = new int[] {6, 7, 8};
int arr3[] = new int[] {9, 10};
int arr4[] = new int[] {11};
int arr5[] = new int[] {12};

root1 = a.createList(arr1);
root2 = a.createList(arr2);
root3 = a.createList(arr3);
root4 = a.createList(arr4);
root5 = a.createList(arr5);

root1.child = root2;
root1.next.next.next.child = root3;
root2.next.child = root4;
root3.child = root5;

root1 = a.flatenEasy(root1);
```

3

3

// flaten a multi level linked list depth wise.



// 1→2→7→11→13→8→12→3